

PROYECTO DE SÍNTESIS: ***CONSTRUCCIÓN DE UN CHATBOT***



Autores: Marc Machado Bou
Xavier Gubau Jové
Centro: INS F. Vidal i Barraquer
Curso: 2023/2024

ÍNDICE

ABSTRACT (English version).....	3
ABSTRACT.....	4
1.- INTRODUCCIÓN.....	5
2.- ESTADO DEL ARTE.....	6
2.1.- ¿Qué es una Inteligencia Artificial?.....	6
2.2.- ¿Qué es un ChatBot?.....	7
2.3.- ¿Qué es una red neuronal?.....	8
2.3.1.- Matriz de confusión: Definición.....	10
2.3.2.- Datos de entrenamiento: Clasificación.....	11
2.4.- ¿Qué es VirtualBox?.....	12
2.5.- ¿Qué es Apache?.....	13
2.6.- ¿Qué son MongoDB y PyMongo?.....	14
3.- CHATBOT: CONSTRUCCIÓN.....	16
PROGRAMA PRINCIPAL:.....	24
Versión web:.....	29
Versión web: Funcionamiento.....	30
Versión web: Sistema de Archivos.....	31
INTERACCIÓN CON EL USUARIO:.....	40
CONTENIDO DESCARTADO:.....	43
Algoritmo genético:.....	43
Integración a la página web del centro:.....	44
4.- RESULTADOS.....	45
POSIBLES MEJORAS:.....	46
5.- CONCLUSIONES.....	47
6.- VALORACIÓN PERSONAL DEL PROYECTO.....	48
BIBLIOGRAFÍA.....	49

ABSTRACT (English version)

In this document, there will be a practical example of how to build a chatbot, similar to the very well-known ChatGPT or the other one who appears to solve questions at the Generalitat de Catalunya's website.

This chatbot has the goal to solve questions about the institute, such as answering doubts about matriculations, taxes, contact information...

Firstly, there is a brief story about the current state of Artificial Intelligences, who are a cutting-edge technology that in a very short time they instaurated in our daily lives, and they can be extremely useful to humanity, because they can make jobs easier or automatize tasks.

Here is explained everything about the model used to develop our project to create the chatbot, and what results are expecting and what methodology we used, which is firstly an investigation and a basic concepts explanation to understand a little bit the algorithm we'll detail later.

Secondly, the results are evaluated and we'll give a personal valoration in reference to them, and lastly, some conclusions and a bibliography of the resources used to elaborate this document.

This project has been elaborated making use of the programming language Python implemented in an Apache web server environment.

ABSTRACT

En este documento, se expondrá un ejemplo práctico de cómo se ha construido un chatbot, similar al ya conocido Chat GPT o el que aparece para resolver dudas en la página web de la Generalitat de Catalunya, el cual tendrá como función el poder resolver dudas referentes al centro, tales como dudas sobre matriculaciones, precios de tasas, información de contacto...

Para comenzar se comenta un poco sobre el estado actual de las Inteligencias Artificiales, las cuales son una tecnología muy puntera que hace relativamente poco que se ha instaurado en nuestra vida cotidiana y pueden ser muy útiles para el ser humano, ya que permiten una automatización y una facilitación de trabajos increíble.

También se habla de que modelo se ha empleado para desarrollar nuestro proyecto para poder crear el chatbot, que resultados esperamos obtener y la metodología que se ha seguido, la cual es primeramente una etapa de investigación y explicación de conceptos básicos para poder comprender un poco el algoritmo que se va a detallar.

Seguidamente se evalúan los resultados y se explicará nuestra valoración personal acerca de estos, y para finalizar, unas conclusiones sobre el proyecto y una bibliografía de recursos que se han empleado para elaborar este documento.

Este proyecto ha sido elaborado haciendo uso del lenguaje de programación Python en conjunto del programa de gestión de servidores web Apache.

1.- INTRODUCCIÓN

Este proyecto tiene como objetivo poner en práctica todo lo aprendido a lo largo del curso; y además poder aprender un poco más sobre la Inteligencia Artificial, ya que, es una herramienta muy poderosa que puede cumplir muchas funciones útiles. Nuestra motivación es querer desarrollar una herramienta que pueda servir de ayuda a futuros alumnos del centro, además de querer superar un nuevo reto como es el programar un chatbot y que este sea funcional.

Otro objetivo que nos hemos propuesto es el de poder crear una base de datos de entrenamiento efectiva, que permita al bot poder dar respuestas con la máxima fluidez posible.

En este proyecto esperamos que el chatbot sea capaz de ofrecer respuestas a preguntas comunes que puedan tenerse a la hora de, por ejemplo, querer matricularse al centro, y que además este bot sea capaz de poder mantener una conversación lo más orgánica posible, algo parecido al chat GPT, una herramienta que en los últimos años ha ganado mucha popularidad por lo potente que es. Otro objetivo que nos hemos propuesto es que este chatbot pueda responder en cualquier idioma.

Nuestra idea a la hora de programar el chatbot es construirlo dentro de un servidor web que estará alojado en una máquina virtual, ya que en anteriores proyectos nos hemos dado cuenta de que a la hora de procesar funciona con bastante rapidez y fluidez.

Esta web será un clon de la web existente del centro (www.vidalibarraquer.net) en la cual se añadirá un desplegable o pop-up que si el usuario hace clic, abrirá el chatbot y podrá hablar con él.

Además, se creará una base de datos haciendo uso de la herramienta MongoDB para que el programa pueda entrenarse con ellos, además de usar las librerías de python de procesamiento de lenguaje (librerías NLTK) y con ello crear un modelo de red neuronal que procesa las preguntas que el usuario le haga, todo esto pudiendo tener la capacidad de detectar el idioma en el que se le está preguntando y/o ofrecer posibilidad de traducir la respuesta.

2.- ESTADO DEL ARTE

2.1.- ¿Qué es una Inteligencia Artificial?

Una Inteligencia Artificial (o IA, para abreviar), es una rama de la informática la cual tiene por objetivo desarrollar algoritmos que permitan a las máquinas el poder tomar decisiones siguiendo los datos facilitados por sus usuarios.

Su desarrollo inicial se remonta a 1956 durante la Conferencia de Dartmouth, donde gracias a John McCarthy, ese evento unió a las mentes más brillantes del momento y se discutió la posibilidad de crear una máquina que tuviera la capacidad de pensar como un ser humano; también hay que añadir que sus conceptos e ideas ya se empezaron a debatir bastante tiempo atrás.

En la década de 1940, ya fue cuando John Von Neumann y Norbert Wiener ya empezaron a trabajar en la teoría de los sistemas y computación que fueron fundamentales para el desarrollo de las IA, más adelante McCarthy definiría el concepto de la Inteligencia Artificial como “la ciencia y la ingeniería de hacer máquinas inteligentes, especialmente programas de ordenador inteligentes”.

Gran parte del “boom” de las IA a lo largo del siglo pasado y este se debe a la ciencia ficción, ya que el concepto de máquinas inteligentes ya apareció por primera vez en obras literarias de 1920, y a partir de ello, han ido haciéndose populares en la cultura del mundo, ejemplos claros serían por ejemplo HAL 9000, de la obra “2001: Una odisea en el espacio” o K.I.T.T de la obra “Knight Rider” también llamado “El Coche Fantástico”¹.



Figura 1: Imagen de HAL 9000²

¹ Fuente: <https://planderecuperacion.gob.es/noticias/que-es-inteligencia-artificial-ia-prtr>

² Fuente: https://en.wikipedia.org/wiki/HAL_9000

2.2.- ¿Qué es un ChatBot?

Un ChatBot, como su nombre indica, es un tipo de IA capaz de generar conversaciones de la forma más orgánica posible, todo esto gracias a la programación que contiene, ya que esta le permite procesar el lenguaje y dar respuestas coherentes a las que el usuario le formula.

Ejemplos históricos de chatbots se remontan a bastantes años atrás, siendo Eliza la primera chatbot creada, desarrollada por Joseph Weizenbaum, esta simulaba que era una psicóloga y se podía mantener una conversación relativamente normal, a día de hoy puede seguirse usando (Ver Figura 2) pero es un programa que está obsoleto.

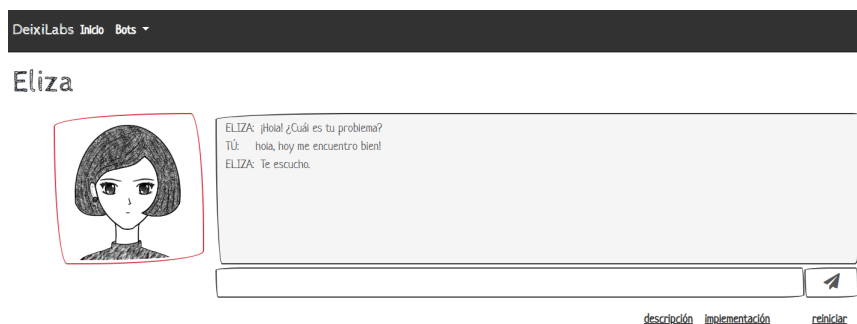


Figura 2: Eliza en la actualidad

Fuente: <http://deixilabs.com/eliza.html>

Más adelante se desarrolló ALICE, que no es más que una versión de Eliza más potente y que tiene mayor capacidad para seguir una conversación.

Y, además, entre medias, se desarrolló el juego Chatterbot en 1989 para tinymud, el cual era un juego de mazmorras el cual tuvo gran éxito porque los usuarios en ningún momento se dieron cuenta de que estaban conversando con una IA³.

Otros Chatbots famosos son Clippy, del paquete de Office pre-2007, donde ya fue descontinuado y eliminado; y Chat GPT, Alexa o Siri.

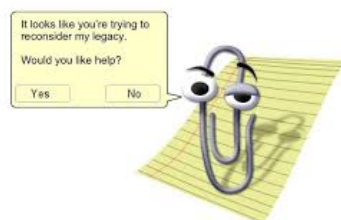


Figura 3: Imagen de Clippy⁴

³ Fuente: <https://automaticchat.com/blog/la-historia-de-los-chatbots>

⁴ Fuente:

<https://www.lavanguardia.com/andro4all/windows/oh-la-nostalgia-el-popular-clippy-vuelve-a-windows-con-mucha-mas-inteligencia>

2.3.- ¿Qué es una red neuronal?

Una red neuronal artificial es un tipo de modelo de Inteligencia Artificial de aprendizaje supervisado; como su nombre indica, es un algoritmo el cual imita las neuronas del cerebro humano.

Este algoritmo consiste en un grupo de unidades llamadas neuronas, las cuales están interconectadas entre sí para transmitir señales.

Todas estas neuronas artificiales interconectadas procesan información de entrada y mediante cálculos con multiplicadores de peso, los cuales hacen que la salida tenga mayor o menor valor, hacen que se genere una salida; esta función se la conoce como función de activación.

Una particularidad de las redes neuronales es su capacidad de aprender y formarse a sí mismas, es decir, no hay ninguna programación en la que intervenga un factor externo para reforzar su aprendizaje⁵.

Aunque sí que puede crearse una matriz de confusión para que el programador pueda saber si la red neuronal funciona correctamente o no.

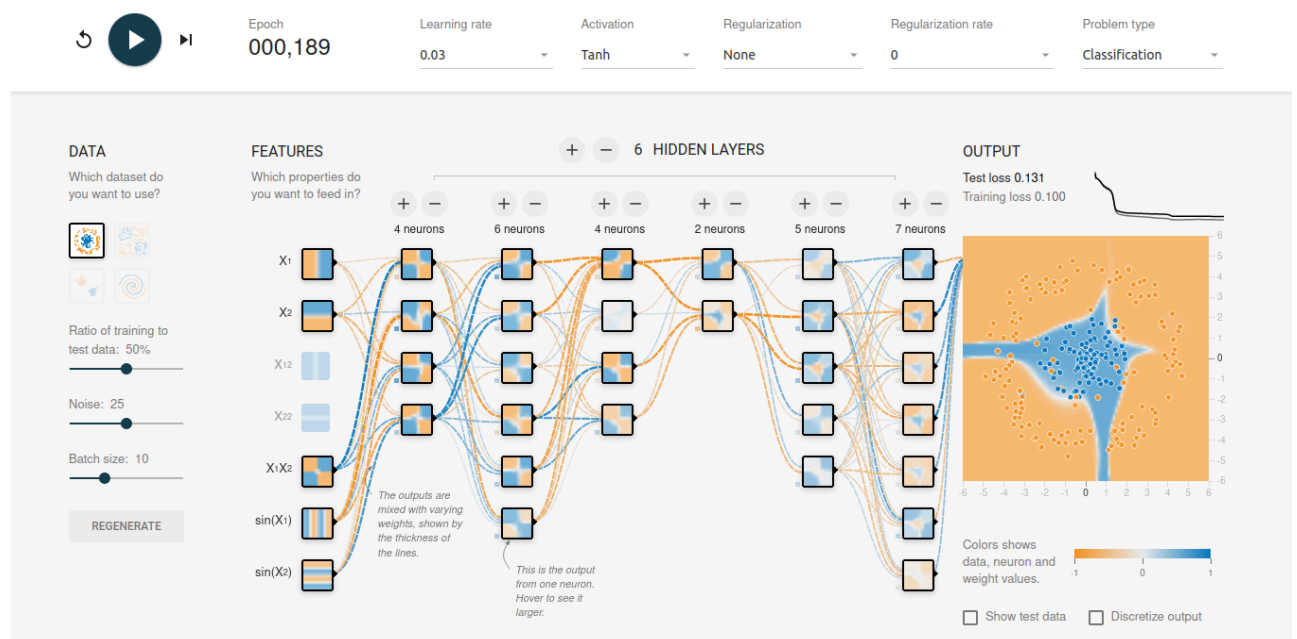


Figura 4: Ejemplo de estructura de red neuronal, usando el simulador de redes neuronales de Tensor Flow⁶

⁵ Fuente: https://es.wikipedia.org/wiki/Red_neuronal_artificial

⁶ Fuente: <https://playground.tensorflow.org/>

La estructura de la red neuronal consiste en un seguido de n entradas, las cuales se empiezan a procesar en la Capa de Entrada, seguidamente, entrando en un número n de capas ocultas, empieza a calcular con pesos y va descartando las salidas que menos peso tienen, es entonces cuando se genera una salida.

Cada neurona también puede tener un número n de ramas, las cuales apuntarán a diferentes perceptrones de las capas ocultas, esto es útil para poder hacer una salida más precisa, aunque cabe destacar que no es directamente proporcional la precisión del programa al número de ramas, a veces menos es más.

Hay que agregar también que el algoritmo puede someterse a reentrenamientos, los cuales pueden mejorar la precisión del programa, ya sea ajustando algún parámetro, el conjunto de datos de entreno o alguna variable, aunque hay que tener cautela, ya que podría originarse problemas de sobreentrenamiento y acabar no siendo preciso.

En este proyecto, se ha hecho uso de una red neuronal de perceptrón múltiple (MLP), la cual es un tipo de red neuronal basada en el uso de múltiples capas (perceptrones), tal y como se muestra en la figura anterior, estas capas, están organizadas de manera jerárquica y cada conexión tiene un peso asociado, este determina que salida dará el programa y hacer que dé la menor cantidad de errores posibles.

2.3.1.- Matriz de confusión: Definición

Una Matriz de confusión es un tipo de algoritmo que permite ver al programador la precisión del modelo de entrenamiento que está construyendo, en este se relacionan los elementos que se procesan en el modelo para ver qué porcentaje de precisión tiene.

La estructura se basa en una cuadrícula de positivo-falso positivo, negativo-falso negativo, tal y como se muestra en la figura:

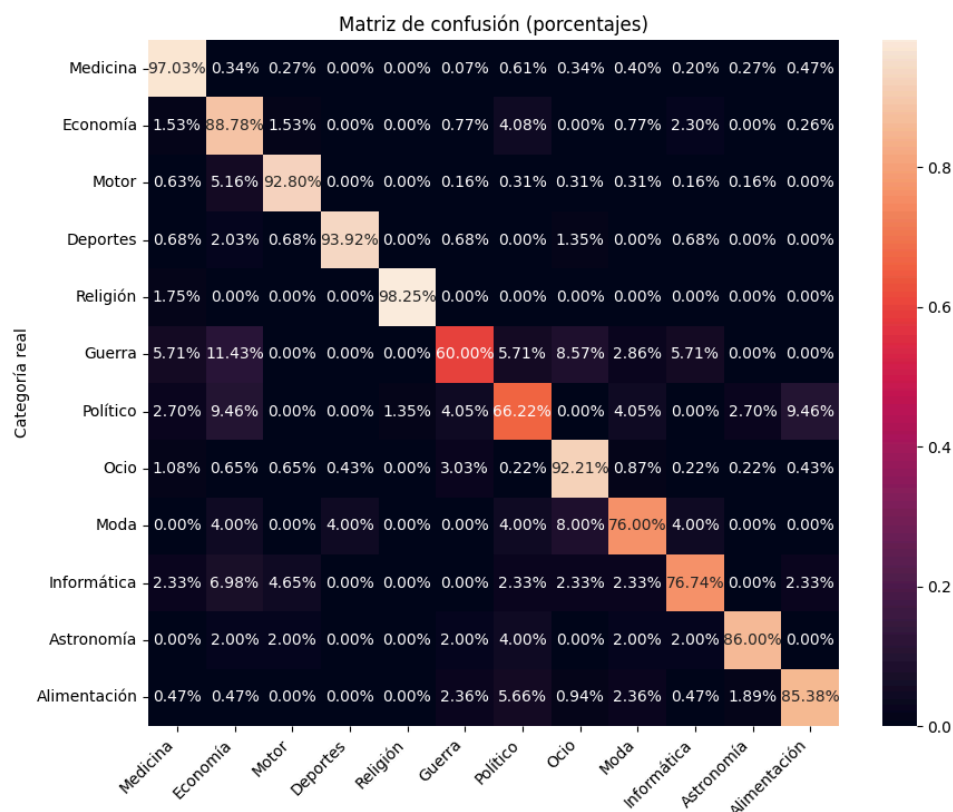


Figura 5: Ejemplo de Matriz de Confusión perteneciente a un clasificador de noticias.

Para cada uno de los casos anteriormente explicados, se puede extraer información que puede ser interesante, como por ejemplo:

- El programa domina perfectamente la clase
- El programa a veces falla en detectar la clase, pero es fiable
- El programa no detecta correctamente la clase.
- El programa detecta la clase de manera correcta pero hay interferencias (ruido) de otras clases.⁷

⁷ Fuente: <https://www.juanbarrios.com/la-matriz-de-confusion-y-sus-metricas/>

2.3.2.- Datos de entrenamiento: Clasificación

Para poder llevar a cabo un algoritmo de red neuronal que funcione de manera correcta, es necesario tener una base de datos de entrenamiento, es decir, una información en la cual el programa pueda sustentarse a la hora de poder procesar datos.

Hay varias maneras de poder darle un conjunto de datos a un programa, las más comunes son un fichero .csv, abreviatura del inglés “Valores separados por comas”, una base de datos en el lenguaje SQL, el cual es un acrónimo del inglés de “Lenguaje de Sentencias Estructuradas”; o también el algoritmo puede “leer” un fichero de texto normal y corriente.

Una vez el programa tiene estos datos, puede clasificarlos de forma automática haciendo uso de ciertas librerías del lenguaje de programación Python, las cuales permiten extraer el contenido del texto, contextualizar y etiquetar a qué categoría pueda pertenecer, todo esto definiendo qué categorías se quieren clasificar, como se muestra en el ejemplo siguiente:

```
# Definir las categorías de clasificación
CATEGORIAS = ['Medicina', 'Economia', 'Motor', 'Deportes', 'Religion',
```

Figura 6: Ejemplo de definición de etiquetas en un algoritmo Python

Seguidamente, con el texto etiquetado y contextualizado, puede procesarse en una red neuronal y, en este ejemplo, generará una salida sobre a qué categoría pertenece el texto, aunque puede aplicarse múltiples usos, en el caso de este proyecto, esto servirá para que el chatbot pueda ofrecer respuestas coherentes.

2.4.- ¿Qué es VirtualBox?

VirtualBox es un programa gestor de máquinas virtuales, dicho de otra manera, permite emular un sistema operativo, esto es útil a la hora de crear algún tipo de servicio web porque facilita mucho su gestión, ya que pueden hacerse copias de seguridad, apagar la máquina en caso de que esta entre en error, se pueden restaurar las copias de seguridad ya hechas...

En resumen, tiene un sinfín de utilidades que para cualquier informático son esenciales.

Para este proyecto, se ha escogido como máquina virtual el sistema operativo Ubuntu 22.04 la cual contendrá una instalación de Apache para poder mostrar el chatbot vía web.

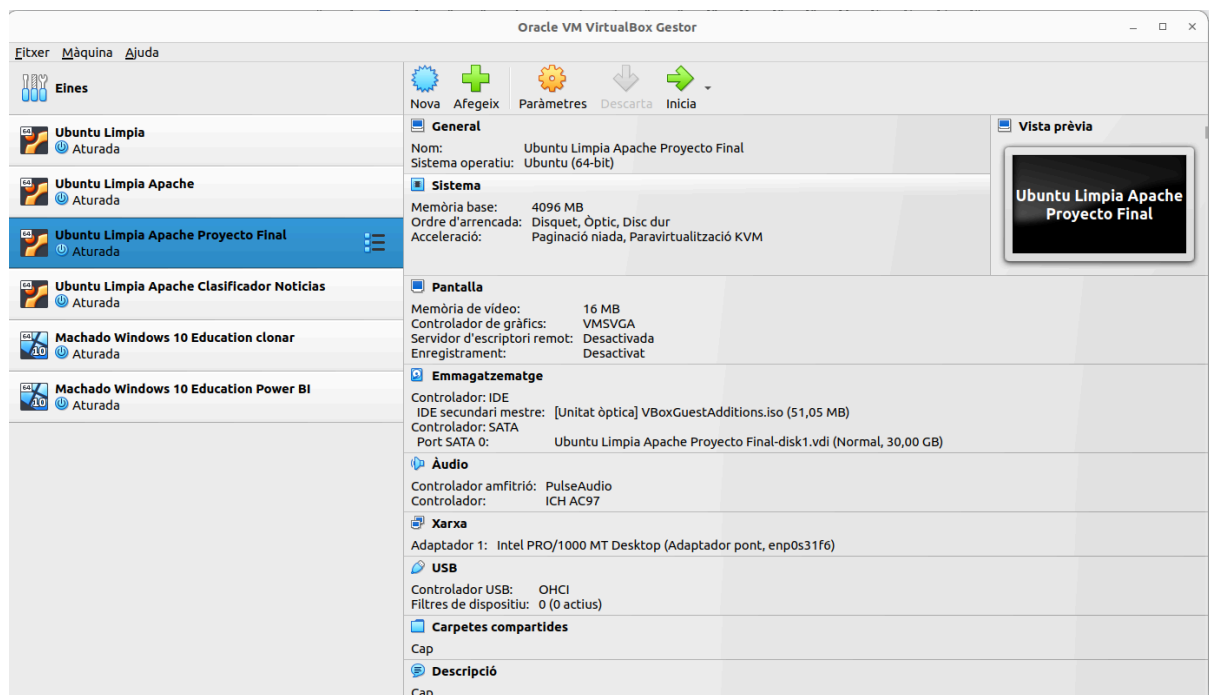


Figura 7: Interfaz del gestor de máquinas virtuales VirtualBox.

2.5.- ¿Qué es Apache?

Apache es un software para crear servidores de protocolo HTTP⁸ de código abierto, es de las herramientas más usadas en el mundo, ya que es muy versátil a la hora de crear un servidor y muy fácil de instalar.

Su funcionalidad consiste en proporcionar todos los recursos y ficheros a todos los usuarios que hagan peticiones a la web, estas solicitudes se llevan a cabo a través de los navegadores web, como Google Chrome o Mozilla Firefox por ejemplo.

Su estructura se basa en módulos, los cuales permiten activar y desactivar funciones a conveniencia del usuario, permitiendo así por ejemplo desactivar que guarde registros de caché, o que no guarda registros (logs) de error⁹.

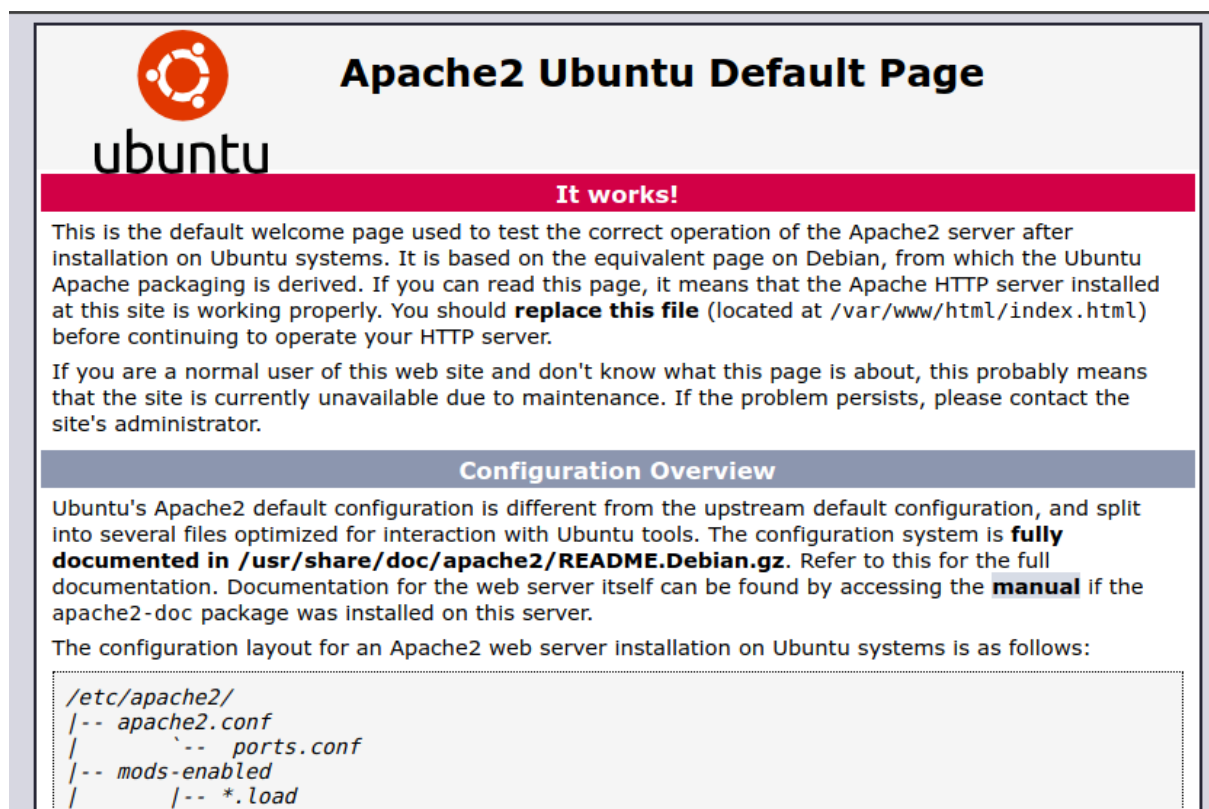


Figura 8: Pantalla por defecto de Apache, se muestra cuando su instalación se termina.

⁸ HTTP es el protocolo de transferencia de Hipertexto, tiene como objetivo el permitir el intercambio de documentos entre servidores.

⁹ Fuente: <https://dinahosting.com/ayuda/que-es-apache-y-para-que-sirve/>

2.6.- ¿Qué son MongoDB y PyMongo?

MongoDB:

MongoDB es un software de código abierto que permite crear bases de datos, extremadamente escalable y flexible, ya que está orientado a gestionar documentos en lugar de bases de datos estructuradas como tal (es decir, por tablas y columnas), esto hace que a la hora de guardar datos sea extremadamente rápido.

Sus principales utilidades consisten en:

- Permitir la búsqueda de ficheros y archivos por campos, rangos y expresiones regulares.
- Permitir crear índices de los documentos; similar a las bases de datos tipo SQL.
- Replicar archivos, es decir, permite guardar copias de seguridad, las cuales no se pueden alterar, de los archivos en diferentes clientes.
- Puede ejecutarse en diferentes servidores para que la carga de datos no cree un cuello de botella en la red, esto hace que sea un entorno muy robusto y con una gran tolerancia a fallos.
- Incluye un framework¹⁰ que permite ejecutar operaciones que se ejecutarán en un entorno de SQL, este framework está construido de tal forma que los datos van pasando a través de diferentes etapas en los cuales son procesados hasta que se consigue la salida que el usuario desee.
- Permite la ejecución del lenguaje de programación JavaScript de manera paralela al servidor¹¹.

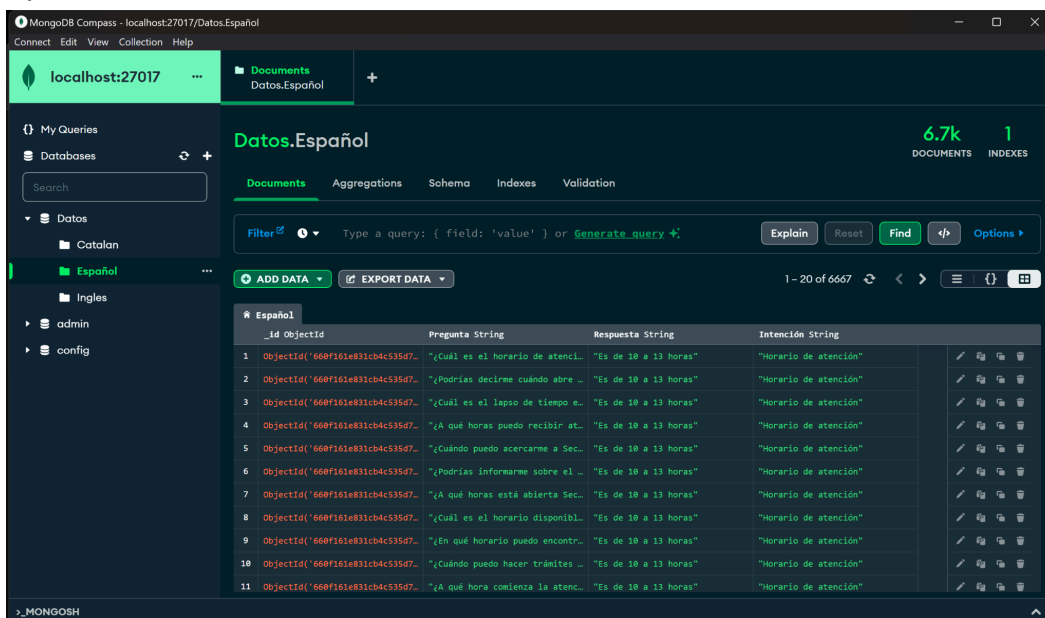


Figura 9: Captura de pantalla de una instalación de MongoDB.

¹⁰ Un framework es un programa que, por lo general, permite interpretar un lenguaje de programación, aunque puede incluir otras funcionalidades.

¹¹ Fuente: <https://es.wikipedia.org/wiki/MongoDB>

PyMongo:

PyMongo es una distribución de Python (librería) que incluye herramientas para integrar MongoDB en Python.

En este proyecto, su uso esencialmente es conectar el modelo de entrenamiento con la base de datos de MongoDB para así poder hacer uso del conjunto de datos de entrenamiento.

```
from pymongo import MongoClient
```

Figura 10: importación de PyMongo en un programa Python.

3.- CHATBOT: CONSTRUCCIÓN

A continuación se detalla como se ha ido desarrollando el proyecto, este se sustenta haciendo uso de dos tipos de modelos de entrenamiento: una máquina de vector de soporte (SVM) y una red neuronal de modelo MLP Classifier (Clasificador Multi Perceptrón).

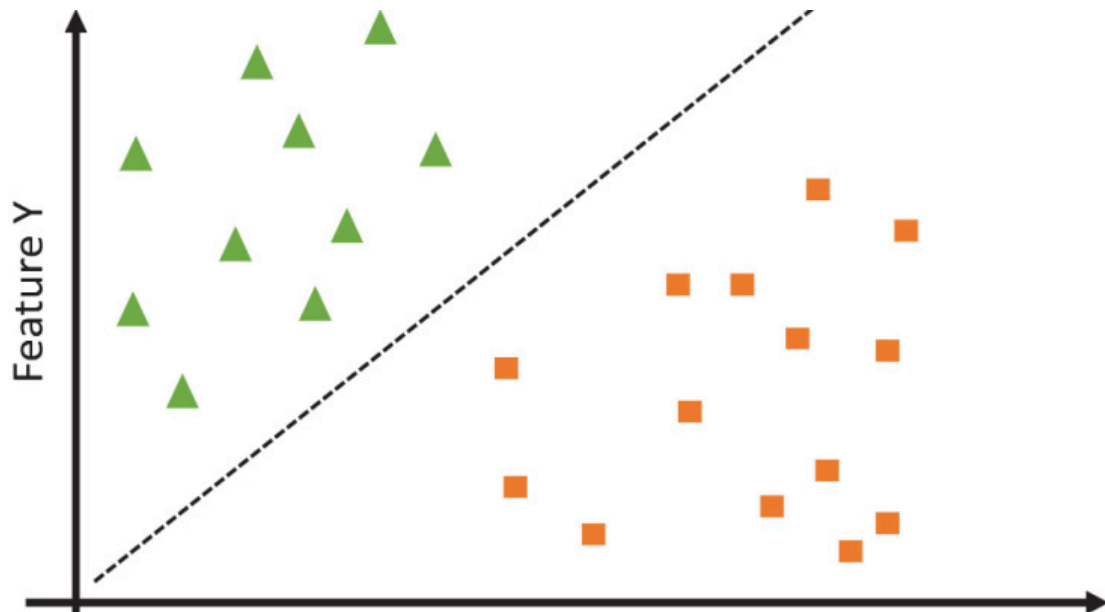


Figura 11: Esquema básico de máquina de vector de soporte.

Una máquina de vector de soporte es un tipo de algoritmo empleado para poder reconocer patrones mediante el uso de regresiones; está lo que hace es, *grosso modo*, identificar varios grupos de datos e intenta predecir a qué categoría pertenecen, estableciendo una frontera entre dos grupos y separándolos buscando el espacio en el que mayor diferencia haya y el margen de error sea cero o lo más pequeño posible.

Recibe el nombre de vector de soporte por los puntos que forman la frontera, los cuales tienen el mismo nombre.

La base de este programa consta de dos archivos Python: un entrenador que crea un archivo binario pickle (.pkl) y el programa principal que lo ejecuta y hace uso de ese archivo creado¹².

Más adelante se detalla como se transfirió todo a Apache y como funciona en un entorno web.

¹² Fuente: https://ca.wikipedia.org/wiki/M%C3%A0quina_de_vector_de_suport

MODELO DE ENTRENAMIENTO:

Conjunto de datos:

Pregunta	Respuesta	Intencion
¿Cuál es el horario de atención al público de Secretaría?	Es de 10 a 13 horas	Horario de atención
¿Podrías decirme cuándo abre Secretaría para atención al público?	Es de 10 a 13 horas	Horario de atención
¿Cuál es el lapso de tiempo en el que puedo visitar Secretaría?	Es de 10 a 13 horas	Horario de atención
¿A qué horas puedo recibir atención en Secretaría?	Es de 10 a 13 horas	Horario de atención
¿Cuándo puedo acercarme a Secretaría?	Es de 10 a 13 horas	Horario de atención
¿Podrías informarme sobre el horario de funcionamiento de Secretaría?	Es de 10 a 13 horas	Horario de atención
¿A qué horas está abierta Secretaría para el público?	Es de 10 a 13 horas	Horario de atención
¿Cuál es el horario disponible para recibir atención en Secretaría?	Es de 10 a 13 horas	Horario de atención
¿En qué horario puedo encontrar personal disponible en Secretaría?	Es de 10 a 13 horas	Horario de atención
¿Cuándo puedo hacer trámites en Secretaría?	Es de 10 a 13 horas	Horario de atención
¿A qué hora comienza la atención al público en Secretaría?	Es de 10 a 13 horas	Horario de atención
¿A partir de qué hora puedo ser atendido en Secretaría?	Es de 10 a 13 horas	Horario de atención
¿Cuál es el horario establecido para visitas en Secretaría?	Es de 10 a 13 horas	Horario de atención
¿A qué hora cierra Secretaría?	Es de 10 a 13 horas	Horario de atención
¿Hasta qué hora está abierta Secretaría?	Es de 10 a 13 horas	Horario de atención
¿Cuándo finaliza la atención al público en Secretaría?	Es de 10 a 13 horas	Horario de atención
¿A qué hora terminan sus labores en Secretaría?	Es de 10 a 13 horas	Horario de atención
¿Cuál es el cierre de jornada en Secretaría?	Es de 10 a 13 horas	Horario de atención
¿A qué hora se suspenden los servicios en Secretaría?	Es de 10 a 13 horas	Horario de atención
¿Cuál es el horario de cierre al público en Secretaría?	Es de 10 a 13 horas	Horario de atención
¿A qué hora concluye el horario de atención en Secretaría?	Es de 10 a 13 horas	Horario de atención
¿Cuándo deja de estar disponible la atención al público en Secretaría?	Es de 10 a 13 horas	Horario de atención
¿Hasta qué hora puedo realizar gestiones en Secretaría?	Es de 10 a 13 horas	Horario de atención
¿Cuándo finaliza el período de atención en Secretaría?	Es de 10 a 13 horas	Horario de atención
¿A qué hora debo llegar para recibir atención en Secretaría?	Es de 10 a 13 horas	Horario de atención
¿Cuándo inicia el horario laboral en Secretaría?	Es de 10 a 13 horas	Horario de atención
¿A qué hora empiezan a recibir al público en Secretaría?	Es de 10 a 13 horas	Horario de atención
¿Cuál es el horario de apertura de Secretaría?	Es de 10 a 13 horas	Horario de atención
¿Cuándo abre sus puertas Secretaría?	Es de 10 a 13 horas	Horario de atención
¿A qué hora comienza el horario de atención al público en Secretaría?	Es de 10 a 13 horas	Horario de atención
¿Cuándo empiezan a atender en Secretaría?	Es de 10 a 13 horas	Horario de atención

Figura 12: Estructura de datos de entrenamiento.

El conjunto de datos que usa el programa es un archivo .csv, en el cual tiene como estructura las preguntas, respuestas e intenciones.

El archivo contiene cerca de diez mil preguntas junto a sus respuestas e intenciones, para obtener el mejor resultado posible.

Primera Versión:

La primera versión del programa es un modelo de entrenamiento que recoge datos de la base de datos de MongoDB haciendo uso de la librería anteriormente explicada llamada PyMongo, tal y como se muestra en la figura siguiente:

```
import pymongo
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.neural_network import MLPClassifier
from nltk.tokenize import word_tokenize
from nltk.stem import WordNetLemmatizer
from tqdm import tqdm
import nltk
import pickle

# Descargar recursos de NLTK (solo necesario una vez)
# nltk.download('punkt')
# nltk.download('wordnet')
# nltk.download('stopwords')

# Definir las stopwords para español
STOPWORDS_CA = set(nltk.corpus.stopwords.words('catalan'))

# Función para cargar los datos desde MongoDB
def cargar_datos_mongodb(coleccion):
    conversaciones = []
    for documento in coleccion.find():
        pregunta = str(documento.get('Pregunta', '')) # Convertir a cadena
        respuesta = str(documento.get('Respuesta', '')) # Convertir a cadena
        conversaciones.append((pregunta, respuesta))
    return conversaciones
```

Figura 13: Carga de datos de MongoDB.

Una vez el programa recoge los datos de la base de datos de Mongo, procede a procesar los textos que encuentra.

```
# Función para preprocesar el texto
def preprocesar_texto(texto):
    # Tokenización
    tokens = word_tokenize(texto)
    # Eliminación de stopwords
    tokens = [word for word in tokens if word.lower() not in STOPWORDS_CA]
    # Lemmatización
    lemmatizer = WordNetLemmatizer()
    tokens = [lemmatizer.lemmatize(word) for word in tokens]
    # Reconstruir el texto preprocesado
    texto_preprocesado = ' '.join(tokens)
    return texto_preprocesado
```

Figura 14: Preprocesamiento de textos.

Con “preprocesar_texto” se refiere al proceso de cortar, eliminar palabras que no aporten nada y añadirle un significado a los textos que encuentre el programa. Una vez el texto preprocesado, se crea y entrena el modelo.

```

# Función para entrenar el chatbot y guardar el modelo y los pesos en un archivo .pkl
def entrenar_chatbot(coleccion, archivo_salida):
    # Cargar los datos
    conversaciones = cargar_datos_mongodb(coleccion)
    if not conversaciones:
        print("No se encontraron datos de conversación.")
        return None

    # Preprocesar los textos
    conversaciones_preprocesadas = [(preprocesar_texto(p), preprocesar_texto(r)) for p, r in conversaciones]

    # Convertir las preguntas y respuestas preprocesadas en listas separadas
    preguntas_preprocesadas, respuestas_preprocesadas = zip(*conversaciones_preprocesadas)

    # Vectorizar los textos de preguntas
    vectorizer = TfidfVectorizer()
    X_preguntas = vectorizer.fit_transform(preguntas_preprocesadas)

    # Crear el chatbot (modelo de clasificación)
    chatbot = MLPClassifier(hidden_layer_sizes=(100,), max_iter=25, random_state=42)

    # Obtener el vector de clases objetivo
    y = list(range(len(conversaciones)))

    # Entrenar el chatbot
    print("Comenzando el entrenamiento del chatbot...")
    chatbot.fit(X_preguntas, y)

    # Entrenar el chatbot con una barra de progreso
    epochs = 25
    for epoch in tqdm(range(epochs - 1), desc="Entrenamiento del Chatbot"):
        chatbot.partial_fit(X_preguntas, y)
    print("Entrenamiento completado.")

```

Figura 15: Función que crea y entrena al chatbot.

Una vez los textos tienen el significado y están con las palabras clave necesarias, el programa empieza a crear el modelo de entreno, tal y como se detalló antes, el modelo que crea es una red neuronal de perceptrón múltiple, primero generando vectores y volviendo a preprocesar textos para una mayor precisión.

Una vez termina, muestra un mensaje de que el entrenamiento ha concluido.

En este primer modelo, se usó simplemente una única capa con 100 perceptrones y 25 iteraciones, cosa que inmediatamente se descartó ya que pese a ser preciso, tenía una carga demasiado lenta.

```

# Guardar el chatbot y los pesos en un archivo .pkl
with open(archivo_salida, 'wb') as f:
    pickle.dump(chatbot, f)
    pickle.dump(vectorizer, f)
    pickle.dump(chatbot.coefs_, f)

# Verificar si los datos se han escrito correctamente en el archivo
print("Revisando el archivo guardado...")
with open(archivo_salida, 'rb') as f:
    saved_chatbot = pickle.load(f)
    saved_vectorizer = pickle.load(f)
    saved_coefs = pickle.load(f)
if saved_chatbot and saved_vectorizer and saved_coefs:
    print("Modelo y pesos guardados en", archivo_salida)
else:
    print("Error al guardar el modelo y los pesos en", archivo_salida)
return chatbot

# Función principal
def main():
    # Conexión a MongoDB
    client = pymongo.MongoClient("mongodb://localhost:27017/")
    # Seleccionar la base de datos
    db = client["Datos"]
    # Seleccionar la colección
    collection = db["Catalan"]
    # Entrenar el chatbot
    archivo_entrenado = "chatbot_entrenado_ca.pkl"
    print("Entrenando el chatbot...")
    chatbot = entrenar_chatbot(collection, archivo_entrenado)
    if chatbot:
        print("El chatbot ha sido entrenado y los pesos han sido guardados correctamente.")

if __name__ == "__main__":
    main()

```

Figura 16: Guardado del programa.

Una vez el programa termina, guarda el modelo en un archivo binario pickle (.pkl) el cual puede usarse más adelante con el programa del chatbot.

Segunda Versión:

```
import pickle
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.metrics import confusion_matrix, classification_report
from nltk.tokenize import word_tokenize
from tqdm import tqdm
from sklearn.preprocessing import LabelEncoder
import re
import unicodedata
import pandas as pd

# Configuración y conexión
DATABASE_NAME = "Datos"
COLLECTION_NAME = "Español"
MODEL_FILENAME = "/media/alumne/HPV212W/CEIABD/Proyecto_Final/ChatBot/chatbot_entrenado_es.pkl"
VECTORIZER_FILENAME = "/media/alumne/HPV212W/CEIABD/Proyecto_Final/ChatBot/tfidf_vectorizer_es.pkl"
LABEL_ENCODER_FILENAME = "/media/alumne/HPV212W/CEIABD/Proyecto_Final/ChatBot/label_encoder_es.pkl"

def load_data_from_csv(csv_file_path):
    """Carga los datos desde un archivo CSV."""
    data = pd.read_csv(csv_file_path)
    preguntas = data['Pregunta'].tolist()
    respuestas = data['Respuesta'].tolist()
    intenciones = data['Intencion'].tolist()
    return preguntas, respuestas, intenciones
```

Figura 17: Carga de datos en la segunda versión.

Para la segunda versión se optó por cambiar la carga de datos de MongoDB a un fichero de valores separados por coma (.csv), ya que ayudaba a que el programa fuera más portable y cargaba los datos un poco más rápido.

```

def train_and_save_model(X, y, model_filename, vectorizer_filename, label_encoder_filename):
    """Entrena el modelo y lo guarda en archivos"""
    #Vectorizar los textos
    vectorizer = TfidfVectorizer()
    X_transformed = vectorizer.fit_transform(X)

    #Codificar etiquetas
    label_encoder = LabelEncoder()
    y_encoded = label_encoder.fit_transform(y)

    #Dividir los datos en conjunto de entrenamiento y prueba
    X_train, X_test, y_train, y_test = train_test_split(X_transformed, y_encoded, test_size=0.2, random_state=42)

    #Ajuste de hiperparametros
    param_dist = {
        'hidden_layer_sizes': [(10,10,10), (10,10,10), (10,10,10), (10,10,10), (10,10,10)],
        'activation': ['relu', 'tanh'],
        'alpha': [0.0001, 0.001, 0.01],
        'solver': ['adam', 'sgd'],
        'max_iter': [1500, 1500, 1500],
    }

    #Crea el modelo de clasificación
    text_clf = MLPClassifier(random_state=42)

```

Figura 18: Entrenamiento del modelo

En esta versión, se puede observar que es un programa mucho más avanzado; pese a no cambiar la función de preprocesar el texto, la cual sigue siendo igual que la anterior, lo que sí cambia es el modelo a crear, ya que hace uso del optimizador Adam, cinco capas con tres perceptrones de 10 ramas cada uno y 1500 iteraciones, además de ajustar los pesos con el parámetro alpha y haciendo uso de dos funciones de activación, modela las entradas y salidas haciendo cálculos con los pesos, relu los hace de forma lineal y tanh calcula una tangente, todo esto para mitigar problemas comunes de las redes neuronales como las “neuronas muertas”, las cuales son neuronas que no varían su peso a lo largo del tratamiento.

Si estas dos funciones no se aplicarán en la red neuronal; sería como la anterior que solo tiene un perceptrón y por tanto, no sería del todo precisa ni eficaz.

```

#Realiza la búsqueda de hiperparámetros mediante búsqueda aleatoria
random_search = RandomizedSearchCV(estimator=text_clf, param_distributions=param_dist, n_iter=10, cv=3, n_jobs=-1)
random_search.fit(X_train, y_train)

#Obtiene el mejor modelo en base a la búsqueda en el csv
best_model = random_search.best_estimator_
print("Mejores hiperparámetros encontrados:")
print(random_search.best_params_)

#Guardar el clasificador, el vectorizador y el LabelEncoder
with open(model_filename, 'wb') as f:
    pickle.dump(best_model, f)

with open(vectorizer_filename, 'wb') as f:
    pickle.dump(vectorizer, f)

with open(label_encoder_filename, 'wb') as f:
    pickle.dump(label_encoder, f)

return best_model, vectorizer, label_encoder

def evaluate_model(X, y, model, vectorizer, label_encoder):
    """Evalúa el modelo y genera una matriz de confusión"""
    # Transformar los textos a características
    X_transformed = vectorizer.transform(X)
    # Predecir las intenciones
    y_pred = model.predict(X_transformed)
    # Convertir las etiquetas predichas a sus valores originales
    y_pred_original = label_encoder.inverse_transform(y_pred)
    # Generar matriz de confusión
    cm = confusion_matrix(y, y_pred_original, labels=label_encoder.classes_)
    print("Matriz de Confusión:")
    print(cm)
    # Generar reporte de clasificación
    report = classification_report(y, y_pred_original, target_names=label_encoder.classes_)
    print("Reporte de Clasificación:")
    print(report)

# Cargar datos
preguntas, respuestas, intenciones = load_data from csv("/media/alumne/HPV212W/CEIABD/Proyecto Final/Preguntas Frecuentes Unidas/Catellano.csv")

```

Figura 19: Función de evaluación en el nuevo modelo.

En esta nueva versión también se implementó una función que busca en el .csv la opción más óptima para dar como salida haciendo uso de los datos ya procesados, más adelante, evalúa que modelo es mejor y entonces lo guarda dentro del archivo binario con pickle.dump.

También, una vez que termina de evaluar, muestra una matriz de confusión con los resultados, tal y como se muestra en la figura siguiente:

```

Preprocesando textos: 100% | 9141/9141 [00:00<00:00, 14801.23it/s]
Mejores hiperparámetros encontrados:
{'solver': 'adam', 'max_iter': 1500, 'hidden_layer_sizes': (10, 10, 10), 'alpha': 0.01, 'activation': 'tanh'}
Matriz de Confusión:
[[ 626   0   0   0   0   0   0   0]
 [   0 461   0   0   0   0   0   0]
 [   0   0 702   0   4   0   1   0]
 [   1   0   0 403   0   0   0   0]
 [   0   0   4 12603   0   2   1   0]
 [   1   0   0   0   0 953   0   0]
 [   0   0   1   0   0   3 1979   0]
 [   0   0   0   1   0   0   2 987   0]
 [   0   0   0   1   0   0   0   0 403]]
Reporte de Clasificación:

```

	precision	recall	f1-score	support
Consecuencias	1.00	1.00	1.00	627
Contacto	1.00	1.00	1.00	461
Costos	0.99	0.99	0.99	707
Documentación	0.99	1.00	1.00	404
Fechas importantes	1.00	1.00	1.00	2611
Horario de atención	1.00	1.00	1.00	954
Información general	1.00	1.00	1.00	1983
Normativas	1.00	1.00	1.00	990
Procedimiento	1.00	1.00	1.00	404
accuracy			1.00	9141
macro avg	1.00	1.00	1.00	9141
weighted avg	1.00	1.00	1.00	9141

Figura 20: Matriz de confusión que genera el programa.

Como se puede observar, el programa se equivoca en pocos apartados, y en los que se equivoca se debe a que hay escasez de datos en el conjunto de entrenamiento, un problema que puede solucionarse fácilmente.

PROGRAMA PRINCIPAL:

Primera versión:

```
1 import os
2 import pickle
3 import random
4 import numpy as np
5 from sklearn.feature_extraction.text import TfidfVectorizer
6 from sklearn.metrics.pairwise import cosine_similarity
7 from nltk.tokenize import word_tokenize
8 from nltk.stem import WordNetLemmatizer
9 from nltk.corpus import stopwords
10 import spacy
11 from spacy.training.example import Example
12 from tqdm import tqdm
13 from pymongo import MongoClient
14
15 # Cargar el modelo entrenado
16 MODEL_FILENAME = "chatbot_entrenado_es.pkl"
17 VECTORIZER_FILENAME = "vectorizer.pkl"
18 LABEL_ENCODER_FILENAME = "label_encoder_es.pkl"
19
20 def load_data_from_mongodb(collection_name):
21     """Carga los datos desde una colección MongoDB."""
22     client = MongoClient('localhost', 27017)
23     db = client['Datos']
24     collection = db[collection_name]
25     data = list(collection.find({}, {'_id': 0}))
26     return data
```

Figura 21: carga de datos de la primera versión del programa.

Como se puede observar, el programa también cargaba los datos desde MongoDB al igual que el entrenador.

```

    return data

def preprocess_text(text, preprocessing=None):
    """Preprocesa el texto, incluyendo stopwords, stemming o lematización"""
    # Tokenización
    tokens = word_tokenize(text)

    # Eliminación de stopwords
    stop_words = set(stopwords.words('spanish'))
    tokens = [word for word in tokens if word.lower() not in stop_words]

    # Aplicar preprocesamiento según la opción
    if preprocessing == 'lemmatization':
        lemmatizer = WordNetLemmatizer()
        tokens = [lemmatizer.lemmatize(word) for word in tokens]

    # Reconstruir el texto preprocesado
    texto_preprocesado = ' '.join(tokens)

    return texto_preprocesado

# Cargar los datos desde MongoDB
data = load_data_from_mongodb("Español")

# Extraer preguntas, respuestas e intenciones
preguntas = [item['Pregunta'] for item in data]
respuestas = [item['Respuesta'] for item in data]

```

Figura 22: Procesamiento de textos en el código antiguo

Los textos los procesa exactamente igual que la primera versión del modelo de entrenamiento; lematizado y tokenizado los textos que encuentra.

```

# Bucle principal para interactuar con el usuario
while True:
    user_input = input("Tú: ")
    if user_input.lower() == "salir":
        break

    # Preprocesar la entrada del usuario
    preprocessed_input = preprocess_text(user_input, preprocessing='lemmatization')

    # Calcular la similitud entre la pregunta del usuario y todas las preguntas de entrenamiento
    X_user = [preprocessed_input]
    vectorizer = TfidfVectorizer()
    X_train_tfidf = vectorizer.fit_transform(preguntas)
    X_user_tfidf = vectorizer.transform(X_user)
    similarities = cosine_similarity(X_user_tfidf, X_train_tfidf)

    # Encontrar el índice de la pregunta más similar
    most_similar_index = np.argmax(similarities)

    # Obtener la respuesta correspondiente a la pregunta más similar
    response = respuestas[most_similar_index]

    print("Chatbot:", response)

```

Figura 23: Programa principal.

El programa principal era muy simple, procesaba la entrada del usuario y le aplicaba un tratamiento similar al que se aplica a la hora de tratar los datos.

Segunda versión:

```
1 from unicode import unicode
2 from sklearn.metrics.pairwise import cosine_similarity
3 from sklearn.pipeline import make_pipeline
4 from sklearn.feature_extraction.text import TfidfVectorizer
5 from sklearn.svm import SVC
6 from nltk.tokenize import word_tokenize
7 from nltk.corpus import stopwords
8 from sentence_transformers import SentenceTransformer
9 from tqdm import tqdm
10 from googletrans import Translator
11 from langdetect import detect
12 from sklearn.decomposition import PCA
13 from annoy import AnnoyIndex
14 import pandas as pd
15 import os
16 import pickle
17
18 MODEL_FILENAME = "/media/alumne/HPV212W/CEIABD/Proyecto_Final/ChatBot/chatbot_entrenado_es.pkl"
19 embedding_cache = {}
20 traducciones_cache = {}
21
22 def remove_accents(input_text):
23     return unicode(input_text)
24
25 def load_data_from_csv(csv_file_path):
26     data = pd.read_csv(csv_file_path)
27     preguntas = data['Pregunta'].tolist()
28     respuestas = data['Respuesta'].tolist()
29     intenciones = data['Intencion'].tolist()
30     return preguntas, respuestas, intenciones
```

Figura 24: Carga de datos en el programa principal.

En esta segunda versión se optó por mejorar el código que ya se tenía, pero el objetivo que se quería lograr era que el chatbot respondiera en cualquier idioma que detectase del usuario, para conseguir esto, se pensó en usar la librería de google traductor, llamada googletrans; la carga de datos y el procesamiento de estos se mantiene igual que en la otra versión.

```

n_components = 16
pca = PCA(n_components=n_components)
model_embedding_reduced = pca.fit_transform(model_embedding.encode(preguntas))

f = model_embedding_reduced.shape[1]
annoy_index = AnnoyIndex(f, metric='angular')
for i, embedding in enumerate(model_embedding_reduced):
    annoy_index.add_item(i, embedding)
annoy_index.build(20)

def get_embedding(text):
    if text in embedding_cache:
        return embedding_cache[text]
    else:
        embedding = model_embedding.encode(text)
        embedding_cache[text] = embedding
        return embedding

translator = Translator()

```

Figura 25: Uso de la librería PCA de Scikit-learn y la librería annoy para optimizar el código.

La primera mejora respecto al código anterior es guardar las consultas a la biblioteca del traductor que ya ha hecho, y después, hacer uso de la biblioteca annoy para calcular los vecinos más cercanos de la salida deseada.

```

def traducir_texto(texto, idioma_destino):
    if (texto, idioma_destino) in traducciones_cache:
        return traducciones_cache[(texto, idioma_destino)]
    else:
        traduccion = translator.translate(texto, dest=idioma_destino).text
        traducciones_cache[(texto, idioma_destino)] = traduccion
        return traduccion

def calcular_similitudes_paralelo(user_embedding):
    indices_vecinos, distancias = annoy_index.get_nns_by_vector(user_embedding, 5, include_distances=True)
    similitudes = [(respuestas[i], 1 - distancia) for i, distancia in zip(indices_vecinos, distancias)]
    return similitudes

```

Figura 26: Funciones de traducción y cálculo de similitudes.

También el código ahora contiene una función de traducción de texto, está lo que hace es detectar el idioma de la entrada del usuario, es decir, si se le escribe algo en inglés, responderá en inglés, y otra función que calcula similitudes y se queda con el mejor resultado.

```

print("Hola! Soy tu Chatbot de poca confianza")

while True:
    user_input = input("Tú: ")
    if user_input.lower() == "salir":
        break

    input_language = detect(user_input)
    translated_input = traducir_texto(user_input, 'es')
    preprocessed_input = preprocess_text(translated_input)
    user_embedding = pca.transform(model_embedding.encode([preprocessed_input]))[0]
    similitudes = calcular_similitudes_paralelo(user_embedding)

    if similitudes:
        respuesta_maxima, porcentaje_maximo = max(similitudes, key=lambda x: x[1])
        respuesta_traducida = traducir_texto(respuesta_maxima, input_language)
        print(f"Respuesta ({porcentaje_maximo * 100}%): {respuesta_traducida}")

        pregunta_feedback = "¿Fue esta respuesta útil? (sí o no):"
        translated_question = traducir_texto(pregunta_feedback, input_language)
        feedback = input(translated_question + " ").lower()
        translated_feedback = traducir_texto(feedback, 'es')

        if translated_feedback.strip().lower() == "si" or translated_feedback.strip().lower() == "sí":
            gracias = traducir_texto("¡Gracias por tu retroalimentación!", input_language)
            print(gracias)

        elif translated_feedback.strip().lower() == "no":
            if len(similitudes) > 1:
                segunda_mejor_respuesta, _ = sorted(similitudes, key=lambda x: x[1], reverse=True)[1]
                translated_second_response = traducir_texto(segunda_mejor_respuesta, input_language)
                print(f"Segunda mejor respuesta: {translated_second_response}")

```

Figura 27: Bucle principal del programa

El bucle principal del programa es muy simple, consta de un bucle infinito, para que el chatbot esté siempre disponible, y funciona de la siguiente manera:

- Primero detecta el lenguaje de la entrada del usuario.
- Seguidamente lo traduce al español para poder tratarlo con los datos de entrenamiento.
- Guarda la consulta traducida para futuros usos
- Procesa el texto.
- Y por último, devuelve la respuesta, ofreciendo una alternativa si el usuario no está satisfecho con el resultado.

Versión web:

Para que el chatbot funcione bien como tal, había que crear un servidor web que sirviera dentro de la red del centro una página web en el que se pudiera conversar con él; para hacer esto, se decidió que lo más idóneo era crear un servidor de Apache, ya que es un software de código abierto muy personalizable y relativamente sencillo de instalar y configurar.

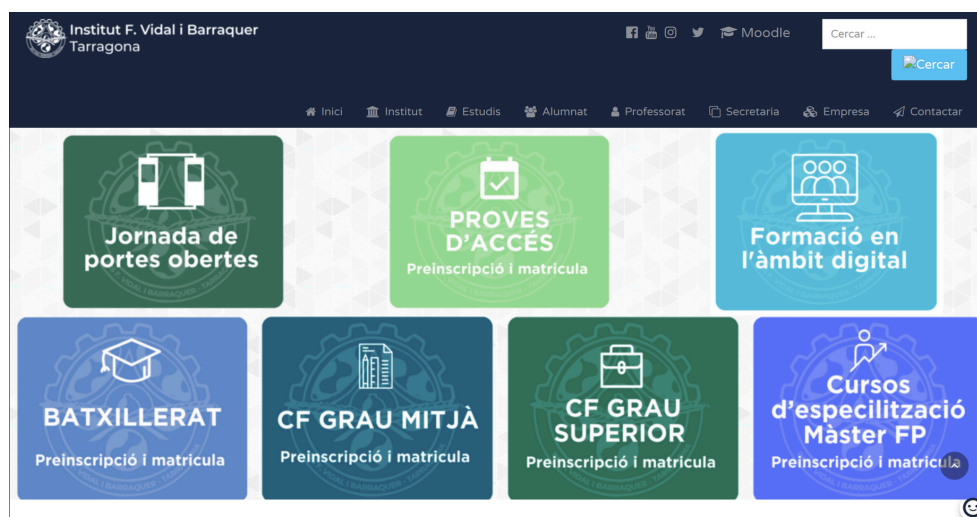


Figura 28: Página web del Chatbot.

La página web que usa el chatbot no es más que una copia de la página web del centro, para que visualmente se vea como quedaría implementado en esta. Como se puede observar, el chatbot aparece como pop-up en la esquina inferior derecha; y para conversar con él solo es necesario hacer clic.



Figura 29: Imagen correspondiente a la segunda versión del Chatbot.

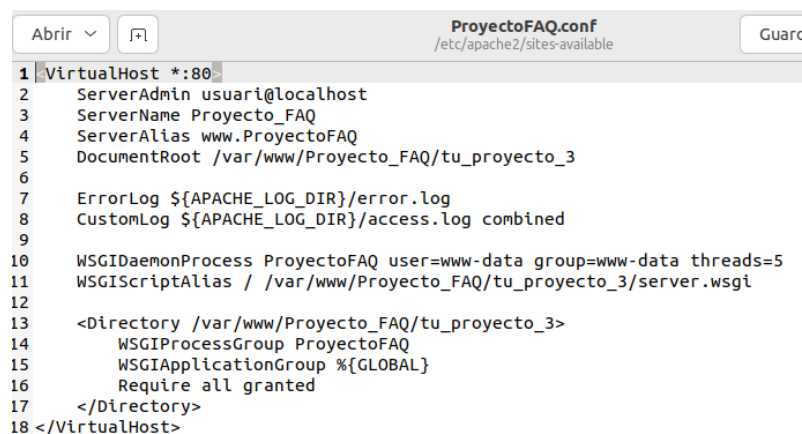
Una vez se le hace clic al pop-up, se despliega una ventana de chat que ya muestra un mensaje de bienvenida.

El primer nombre que se le dió, VidalKiller fue por una competición que había con unos compañeros del aula, ya que su bot se llamaba Vidalet, entonces lo llamamos así, aunque más adelante se le cambió el nombre a otro mejor.

Versión web: Funcionamiento.

Para que el Chatbot funcione, como se mencionó en apartados anteriores, había que implementarlo en un servidor Apache, a continuación se detalla todo lo que hay configurado para que funcione correctamente.

Es importante añadir que Apache es un software que hace gran uso de los permisos del sistema Linux, entonces es importante tenerlo en cuenta cada vez que se reemplace un archivo o se haga cualquier cambio.

A screenshot of a text editor window showing the configuration file 'ProyectoFAQ.conf' located at '/etc/apache2/sites-available'. The file contains an Apache VirtualHost configuration for port 80. The configuration includes settings for the server admin, name, alias, document root, error logs, custom logs, WSGI daemon process, and WSGI script alias. It also includes a directory configuration for the project's web root with WSGI process and application group settings, and a 'Require all granted' directive.

```
1 VirtualHost *:80
2     ServerAdmin usuari@localhost
3     ServerName Proyecto_FAQ
4     ServerAlias www.ProyectoFAQ
5     DocumentRoot /var/www/Proyecto_FAQ/tu_proyecto_3
6
7     ErrorLog ${APACHE_LOG_DIR}/error.log
8     CustomLog ${APACHE_LOG_DIR}/access.log combined
9
10    WSGIDaemonProcess ProyectoFAQ user=www-data group=www-data threads=5
11    WSGIScriptAlias / /var/www/Proyecto_FAQ/tu_proyecto_3/server.wsgi
12
13    <Directory /var/www/Proyecto_FAQ/tu_proyecto_3>
14        WSGIProcessGroup ProyectoFAQ
15        WSGIApplicationGroup %{GLOBAL}
16        Require all granted
17    </Directory>
18 </VirtualHost>
```

Figura 30: archivo de configuración principal del proyecto.

Es necesario que en el directorio sites-available de Apache haya un archivo de configuración en el cual estén registrados todos los directorios del proyecto, ya que Apache será incapaz de localizarlos y entrará en estado de error; una vez hecho esto, simplemente hay que activar el archivo con el comando “sudo a2ensite ProyectoFAQ.conf”.

Una vez hecho esto, ya Apache servirá únicamente a este archivo.

Versión web: Sistema de Archivos.

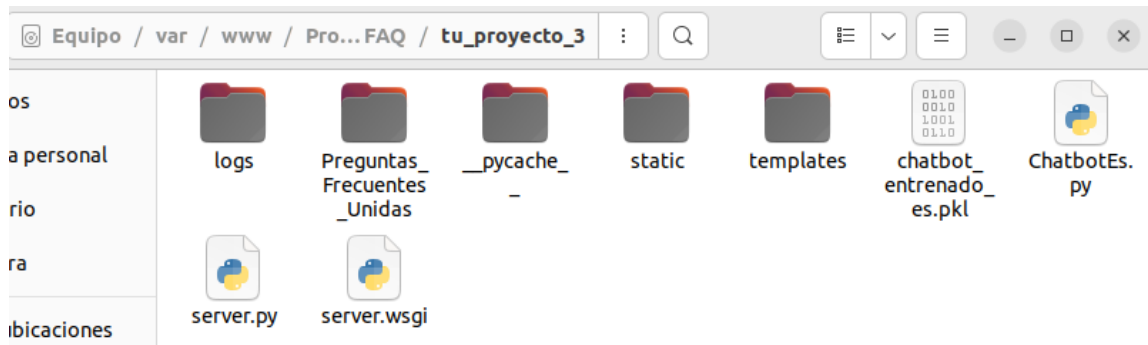


Figura 31: Sistema de archivos del proyecto.

Para que el sistema funcione correctamente, requiere de tener un directorio con todos los archivos necesarios.

Grosso modo, los archivos importantes a detallar son los siguientes:

server.wsgi:



Figura 32: Captura de server.wsgi

Un archivo .wsgi (o **Web Server Gateway Interface**) es un tipo de archivo que le sirve a Apache para poder ejecutar aplicaciones escritas en Python, en este caso, la aplicación en cuestión es una aplicación Bottle.

Para que funcione correctamente, Apache debe tener instalado y habilitado mod_wsgi.

server.py:

```
from bottle import TEMPLATE_PATH
TEMPLATE_PATH.append('/var/www/Proyecto_FAQ/tu_proyecto_3/templates')

@app.route('/chatbot', method='POST')
def chatbot():
    """Handles the chatbot POST request."""
    try:
        siguientes_mejores_respuestas = []

        data = json.loads(request.body.read())
        user_input = data.get('user_input')
        if not user_input:
            return {"bot_response": "Por favor, ingresa algo para conversar."}
        input_language = detect(user_input) if user_input is not None else 'en'

        if input_language != 'es':
            translated_input = traducir_texto(user_input, 'es')
        else:
            translated_input = user_input

        preprocessed_input = preprocess_text(translated_input)

        user_embedding = pca.transform(model_embedding.encode([preprocessed_input]))[0]

        similitudes = calcular_similitudes_paralelo(user_embedding)
        logging.info(f"User query: {user_input}")

        respuestas_candidatas = [(respuestas[preguntas.index(pregunta)], similitud) for
pregunta, similitud in similitudes if similitud > 0.2]

        if respuestas_candidatas:
            respuesta, _ = max(respuestas_candidatas, key=lambda x: x[1])
            logging.info(f"Best response: {respuesta}, Similarity: {max(similitudes,
key=lambda x: x[1])[1]}")
        else:
            if len(similitudes) == 0:
                respuesta = "¿Podrías reformular tu pregunta? No encuentro información
relacionada."
            else:
                respuesta = "No estoy seguro de entender. ¿Podrías proporcionar más
detalles?"
            logging.warning("No suitable response found.")

        translated_response = traducir_texto(respuesta, input_language)

        https_match = re.search(r"https?:\/\/[^\s]+", translated_response)
        if https_match:
            link_text = https_match.group(0)
            respuesta = f"{respuesta} ({urllib.parse.quote(link_text, safe='')})"
```

```

        if user_input.lower().strip() == "jarl 9000":
            image_url = "/static/jarl9000.png"
            response_text = "¡Eres un fistro! Jarl 9000 al atacar!l, cobarde."
            with open(os.path.join("/var/www/Proyecto_FAQ/tu_proyecto_3/static",
"jarl9000_sound.mp3"), 'rb') as f:
                sound_content = f.read()
            return {"bot_response": response_text, "image_url": image_url,
                    "sound_url": "/static/jarl9000_sound.mp3",
                    "user_embedding": user_embedding.tolist(), "similitudes": similitudes,
                    "input_language": input_language, "next_best_responses":
siguientes_mejores_respuestas}

        return {"bot_response": translated_response, "user_embedding":
user_embedding.tolist(),
                "similitudes": similitudes, "input_language": input_language,
                "next_best_responses": siguientes_mejores_respuestas}
    except json.JSONDecodeError:
        return {"bot_response": "Invalid data format."}
    except Exception as e:

        print(f"An error occurred: {e}")
        return {"bot_response": "Ocurrió un error. Por favor, inténtalo de nuevo más
tarde.",
                "next_best_responses": []}

```

Figura 33: Método de chatbot

El código Python del programa no varía del anterior, sin embargo, para que funcione en el entorno web, hay que tener programados métodos para que bottle pueda ejecutarlos, este método, se encarga de cargar todo el código python anterior, además de que trata cualquier posible error que pudiera ocurrir.

Su funcionamiento se basa en ofrecer como primera respuesta la mejor que encuentre después de tratar y procesar la entrada del usuario, y en el caso de que no la entienda, pida que se repita

También, estos métodos están vinculados a un script de JavaScript, alojado en el directorio de archivos estáticos, para que funcionen al unísono.

```

feedback_negativo_count = defaultdict(int)
@app.route('/feedback', method='POST')
def feedback():
    try:
        data = json.loads(request.body.read())
        translated_feedback = data.get('feedback')
        user_embedding = np.array(data.get('user_embedding'))
        similitudes = data.get('similitudes')
        input_language = data.get('input_language')
        logging.info(f"User feedback: {translated_feedback}")
        if translated_feedback.strip().lower() == "si" or
translated_feedback.strip().lower() == "sí":
            gracias = traducir_texto(";Gracias por tu retroalimentación!", input_language)
            return {"message": gracias}
        elif translated_feedback.strip().lower() == "no":
            feedback_negativo_count[input_language] += 1
            if feedback_negativo_count[input_language] >= 3:
                mensaje_limite = traducir_texto("Si no he solucionado tu problema o pregunta
llama al 977 21 28 36.", input_language)
                return {"message": mensaje_limite}
            else:
                if len(similitudes) > 1:
                    similitudes_ordenadas = sorted(similitudes, key=lambda x: x[1],
reverse=True)
                    segunda_mejor_similitud = similitudes_ordenadas[1]
                    pregunta_segunda_mejor_respuesta = segunda_mejor_similitud[0]
                    actual_response =
respuestas[preguntas.index(pregunta_segunda_mejor_respuesta)]
                    translated_second_response = traducir_texto(actual_response,
input_language)
                    return {"message": translated_second_response}
                else:
                    no_similitudes = traducir_texto("No se encontraron más similitudes.",
input_language)
                    return {"message": no_similitudes, "next_best_responses": []}

    except Exception as e:
        print(f"An error occurred: {e}")
        return {"message": "Ocurrió un error al procesar tu solicitud.",
"next_best_responses": []}

```

Figura 34: Método de feedback.

También se configuró un método para poder dar retroalimentación, en este caso lo que hace es ofrecer respuestas alternativas a la que dió, y en el caso que el usuario no esté satisfecho con tres de las respuestas, ofrece una cuarta respuesta para poder ponerse en contacto con el centro, también tiene configurado que vaya traduciendo las respuestas a la lengua detectada, así se garantiza que siempre responda en la lengua que detecte.

```

@app.route('/translate', method='POST')
def translate_text():

```

```

try:
    data = json.loads(request.body.read())
    text = data.get('text')
    target_language = data.get('target_language')
    translated_text = traducir_texto(text, target_language)

    return {"translated_text": translated_text}

except Exception as e:
    print(f"An error occurred during translation: {e}")
    return {"error": "An error occurred during translation."}

```

Figura 35: Método de traducción.

El último método referente al código que es necesario mencionar es el método de traducción, este se encarga de cargar la función de traducir el texto del código Python, y si hay algún error mientras traduce, muestra un mensaje.

```

@app.route('/')
def index():
    return template('index.html')

@app.route('/static/<filename:path>')
def serve_static(filename):
    return static_file(filename,
root='/var/www/Proyecto_FAQ/tu_proyecto_3/static')

```

Figura 36: Métodos para servir el directorio estático y de plantillas.

Estos métodos, pese a ser pequeños, son los más importantes, ya que se encargan de hacer que el programa sirva la página web y que cargue los archivos estáticos.

Directorio Static:

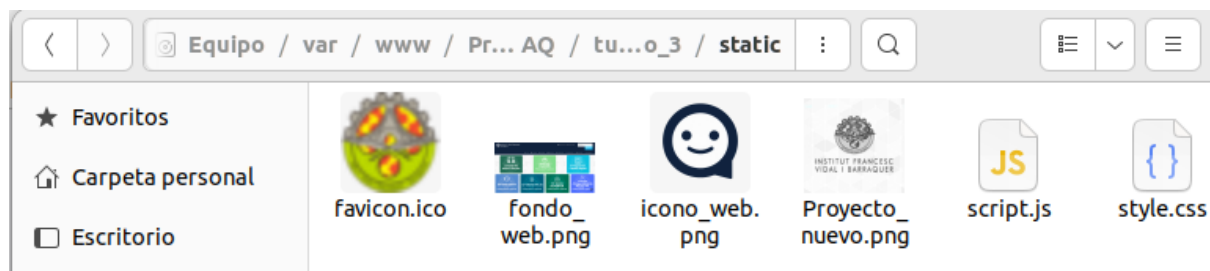


Figura 37: Directorio de archivos estáticos

Este directorio contiene todos los archivos estáticos necesarios para que el chatbot y la web funcionen en consonancia, con el script de JavaScript el cual controla todas las acciones que hace el chatbot, como por ejemplo, ejecutar el programa server.py, controla si el usuario envía retroalimentación, llama a la hoja de estilos css...

Script.js:

```
fetch("/chatbot", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
  },
  body: data,
})
.then((response) => {
  if (!response.ok) {
    throw new Error("Error: Network response was not ok");
  }
  return response.json();
})
.then((response) => {
  handleServerResponse(response);
})
.catch((error) => {
  console.error("Error:", error);
  const errorMessage = document.createElement("p");
  errorMessage.textContent = `Error: ${error.message}`;
  chatArea.appendChild(errorMessage);
});
```

Figura 38: Función fetch del chatbot en el script de Js.

Como se mencionó antes, el script de JavaScript está pensado para funcionar en consonancia con el script de Python server.py, para lograr esta función, se hace con la API fetch, está lo que hace es buscar en el código Python todas las rutas configuradas en los métodos.

Esto se hace para cada método que implique una interacción con el usuario, como por ejemplo el método /feedback.

```

}
return fetch('/feedback', {
  method: 'POST',
  headers: {
    'Content-Type': 'application/json'
  },
  body: JSON.stringify({
    'feedback': feedback,
    'user_embedding': data.user_embedding,
    'similitudes': data.similitudes,
    'input_language': data.input_language,
    'negative_feedback_count': negativeFeedbackCount,
    'conversation_id': 123
  })
})
})
.then(response => {
  if (!response.ok) {
    throw new Error("Error en la respuesta del servidor");
  }
  return response.json();
});

```

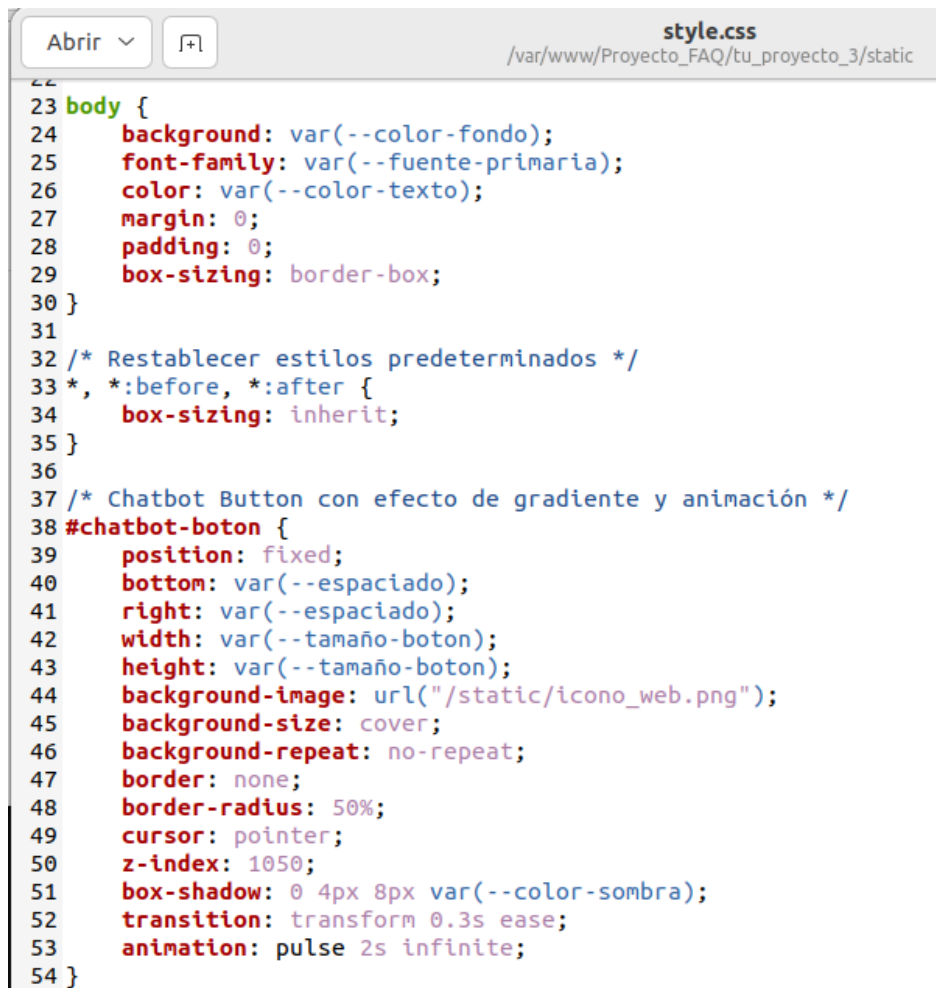
Figura 39: Función fetch para el feedback

En esta función, se encarga de procesar que el usuario haga click en los pulgares hacia arriba o hacia abajo en el chat y mandar una respuesta al servidor en consecuencia.

En caso de mandar un no, se ejecuta la función feedback anteriormente mostrada en el código Python.

También el script permite crear otras funcionalidades, tales como ejecutar el chat cuando el usuario hace clic en el pop-up, crear los contenedores de mensajes en él.

Style.css:



```
23 body {
24     background: var(--color-fondo);
25     font-family: var(--fuente-primaria);
26     color: var(--color-texto);
27     margin: 0;
28     padding: 0;
29     box-sizing: border-box;
30 }
31
32 /* Restablecer estilos predeterminados */
33 *, *:before, *:after {
34     box-sizing: inherit;
35 }
36
37 /* Chatbot Button con efecto de gradiente y animación */
38 #chatbot-boton {
39     position: fixed;
40     bottom: var(--espaciado);
41     right: var(--espaciado);
42     width: var(--tamaño-boton);
43     height: var(--tamaño-boton);
44     background-image: url("/static/icono_web.png");
45     background-size: cover;
46     background-repeat: no-repeat;
47     border: none;
48     border-radius: 50%;
49     cursor: pointer;
50     z-index: 1050;
51     box-shadow: 0 4px 8px var(--color-sombra);
52     transition: transform 0.3s ease;
53     animation: pulse 2s infinite;
54 }
```

Figura 40: Captura de la hoja de estilos CSS.

La función de la hoja de estilos CSS es la de agregar efectos especiales, cambiar colores, ajustar el tamaño del chatbot, establecer la imagen de este, entre otras funcionalidades, en resumen, se encarga del apartado visual del proyecto.

Directorio Templates:



Figura 41: Directorio de plantillas (templates).

El directorio templates contiene el archivo .html de la web, este se encarga de cargar el script de JavaScript y de mostrar la página tal y como se ve en el apartado anterior.

Index.html:

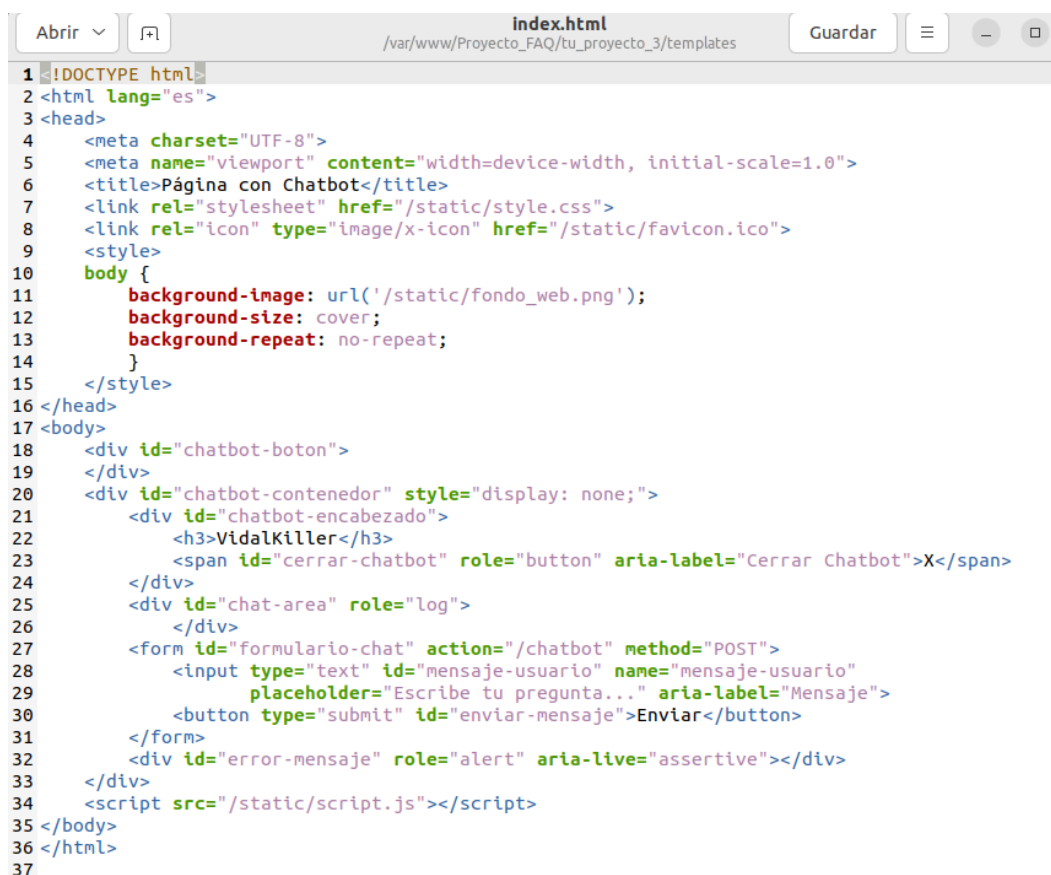


Figura 42: Index.html

El documento que controla la web es muy simple, carga la hoja de estilos, el fondo y el chatbot, que no es más que un formulario html.

INTERACCIÓN CON EL USUARIO:

Primera versión:

A lo largo del desarrollo, el chatbot ha ido sufriendo cambios visuales, aquí se detallan los más notorios:

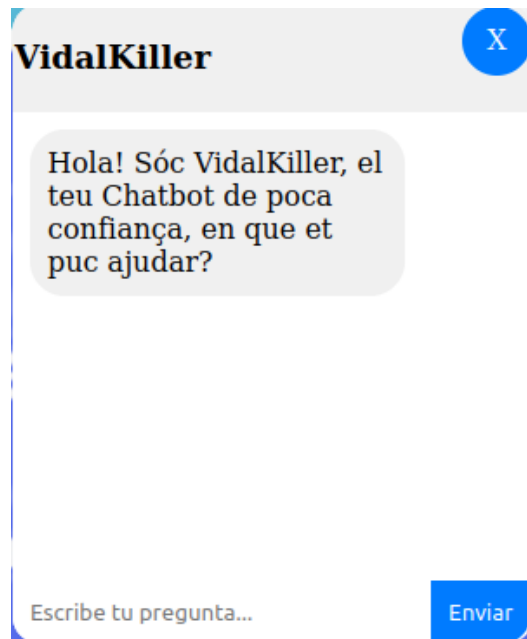


Figura 43: Primera versión del chatbot.

El primero tenía una interfaz más similar a la de messenger, sin embargo, no terminaba de convencer por lo simple que se ve, entonces se decidió que se haría algo un poco más profesional.

Segunda versión:



Figura 44: Segunda versión del chatbot.

La siguiente versión se optó por algo más sofisticado y elegante, con unos colores y fondo que encajaran más con la web del centro, aunque aún se seguía llamando VidalKiller.

También se implementaron algunas mejoras como que conforme la conversación fuese avanzando, hubiera un auto-scroll hacia abajo.

Versión final:



Figura 45: Captura del guiño a Chiquito de la Calzada que se añadió en la versión final

La versión final varía muy poco estéticamente respecto a la versión anterior, pero ya se decidió su nombre definitivo, Jarl 9000, que básicamente sería como la versión española de HAL 9000, pero con un homenaje a Chiquito de la Calzada. Se implementó un pequeño guiño en el que sí se menciona en el chat su nombre, muestra una frase y da la opción de reproducir una pista de audio.

CONTENIDO DESCARTADO:

En este proyecto se hicieron pruebas con otros algoritmos, a continuación se detallan algunos que no funcionaron correctamente o que directamente no funcionaban:

Algoritmo genético:

El primero que se probó fue el implementar un algoritmo genético para que el chatbot respondiera de forma más “humana” por así decirlo, sin embargo lo que no sabíamos es que estábamos programando a cierto político español.

```
def fitness_function(query):
    query_embedding = sentence_model.encode([query])
    similarities = cosine_similarity(query_embedding, sentence_embeddings).flatten()

    # Get indices of top-3 most similar questions
    top_k_indices = similarities.argsort()[-3:][::-1]

    # Get intentions of retrieved questions
    retrieved_intentions = [intentions[i] for i in top_k_indices]

    # Calculate score based on the frequency of the most common intention
    most_common_intention = max(set(retrieved_intentions), key=retrieved_intentions.count)
    intention_score = retrieved_intentions.count(most_common_intention) / len(retrieved_intentions)

    # Combine similarity and intention score
    combined_scores = similarities + intention_score
    return combined_scores.max()

def mutate_query_thesaurus(query):
    words = query.split()
    i = random.randint(0, len(words) - 1)
    synonyms = wordnet.synsets(words[i])
    if synonyms:
        synonym = random.choice(synonyms).lemmas()[0].name()
        words[i] = synonym
    return " ".join(words)

def crossover_queries(query1, query2):
    split_point = random.randint(1, min(len(query1.split()), len(query2.split()))) - 1
    return " ".join(query1.split()[:split_point] + query2.split()[split_point:])
```

Figura 46: Estructura del algoritmo genético

Este algoritmo se basaba en aplicar la estrategia crossover, es decir, quedarse con los mejores resultados generados y sustituir a los que eran menos precisos.

```

Ask a question or provide a topic (or type 'exit'): que tengo que hacer para ponerme en cont
acto con el tutor de mi hijo?
Detected intention: Horario de atención
The attention mask and the pad token id were not set. As a consequence, you may observe unex
pected behavior. Please pass your input's `attention_mask` to obtain reliable results.
Setting `pad_token_id` to `eos_token_id`:50256 for open-end generation.
Generated Text: Intención: Horario de atención
Contexto: Pregunta similar: ¿Podrías proporcionarme el enlace digital que abarca todos los a
spectos de preinscripción y matrícula?
Intención: Horario de atención
Respuesta: Es de 10 a 13 horas

Pregunta similar: ¿Podrías proporcionarme el enlace digital que abarca todos los aspectos de
preinscripción y matrícula?
Intención: Horario de atención
Respuesta: Es de 10 a 13 horas

Pregunta similar: ¿Podrías proporcionarme el enlace digital que abarca todos los aspectos de
preinscripción y matrícula?
Intención: Horario de atención
Respuesta: Es de 10 a 13 horas

Texto generado: Ángel en el público de la puebla de los poblaciones de las poches de línea,
porque está en la dellación, que esta es una página, con el años, en este puede, es esto en
los año, el que señor, de según de todo, y estos que no seguido, no estar a poco, o que toda
s, a la ciudad de uno, se hace, haciendo, estas que hacenas. ¡Porquemos están con la enliven
za de sus pobres, para la porquista con los enciñas en las Pueblas de Los Piedras de San Jua
n de Pineda.
Ask a question or provide a topic (or type 'exit'): exit

```

Figura 47: Resultado del algoritmo genético.

Como se puede observar, el texto que genera carece de coherencia alguna, por ende acabó siendo descartado sin pasar la primera fase del desarrollo, ya que todos los textos que respondía eran así.

Integración a la página web del centro:

Otro contenido descartado fue el descargar la página web del centro e intentar integrarlo en ella para ver como quedaba el pop-up y como cargaba el chat, sin embargo, al cargarla en el servidor, había conflicto entre la web del centro y el servidor, por ende, acabó siendo descartado.

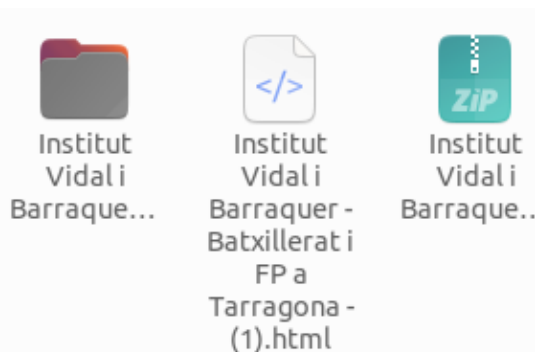


Figura 48: Restos del contenido descartado.

Lo único restante que queda actualmente son tres archivos de la web del instituto.

4.- RESULTADOS

Para probar la funcionalidad del chatbot, se aplicará un gold standard que se muestra una porción de ello en la siguiente tabla:

Pregunta	OK ₁	OK _{2/3}	sin info / con sentido	NOK
¿Cual es el horario de secretaria?	1	0	0	0
¿A que hora abre el centro?	1	0	0	0
¿A partir de que hora puedo entrar al instituto?	1	0	0	0
¿En que direccion esta el instituto?	0	0	0	1
¿Puedo aparcar cerca del instituto?	1	0	0	0
¿Que numero he de marcar para llamar al centro?	0	0	0	1
¿Cuando cierra el instituto?	1	0	0	0

Figura 49: Muestra de gold standard.

Aquí solo se muestra un ejemplo de 7 preguntas, el conjunto total son 50.

TOTAL	40	6	2	52
--------------	----	---	---	----

Figura 50: Resultados de gold standard.

Se puede observar que de las 50 preguntas ha acertado un 40% de ellas a la primera, un 6% a la segunda, y un 2% de ellas ha respondido sin información pero tiene sentido la respuesta ofrecida, ya sea porque proporciona un enlace hacia el sitio que se le pregunta o da una información que tiene correlación con el tema preguntado, frente a un 52% de fallo.

En resumen, tiene un porcentaje de fallo bastante elevado, pero se puede mejorar de las maneras que se detallan en la siguiente sección.

POSIBLES MEJORAS:

Pese a que el proyecto es relativamente preciso, hay que tomar en cuenta que no es perfecto, ya que pese a que el dataset es un conjunto de más de diez mil líneas, es un conjunto pequeño de datos para hacer funcionar bien un chatbot, así que, una mejora sería ampliar aún más el dataset.

Otra mejora posible es integrarlo a la web del centro, pese a que se intentó hacer, no se pudo debido a la cantidad de problemas que dió.

También otra propuesta de mejora es integrar al servidor a la aplicación MongoDB, que pese a que en local si funciona; en el servidor se intentó configurar y no funcionó.

Otra mejora es añadir que se pueda despedir del usuario en cualquier idioma, ya que el único idioma que trata en la despedida es el español.

Algo más que se podría mejorar es tener aún más en consideración que hay muchas palabras en castellano que poseen el mismo significado, siendo un ejemplo común, cantina y cafetería, y esto recae en lo anteriormente mencionado sobre que es un dataset “pequeño”.

5.- CONCLUSIONES

Hoy por hoy, este ha sido un reto muy desafiante, y a lo largo del transcurso del desarrollo hemos encontrado multitud de problemas, como por ejemplo, errores pequeños que no se ven a simple vista, como es por ejemplo un espacio que no debería de estar; errores en que a veces Apache no encontraba el directorio, problemas de permisos de archivos... En definitiva, una gran amplitud de problemas técnicos, sin embargo, hemos conseguido los objetivos que nos propusimos, los cuales eran:

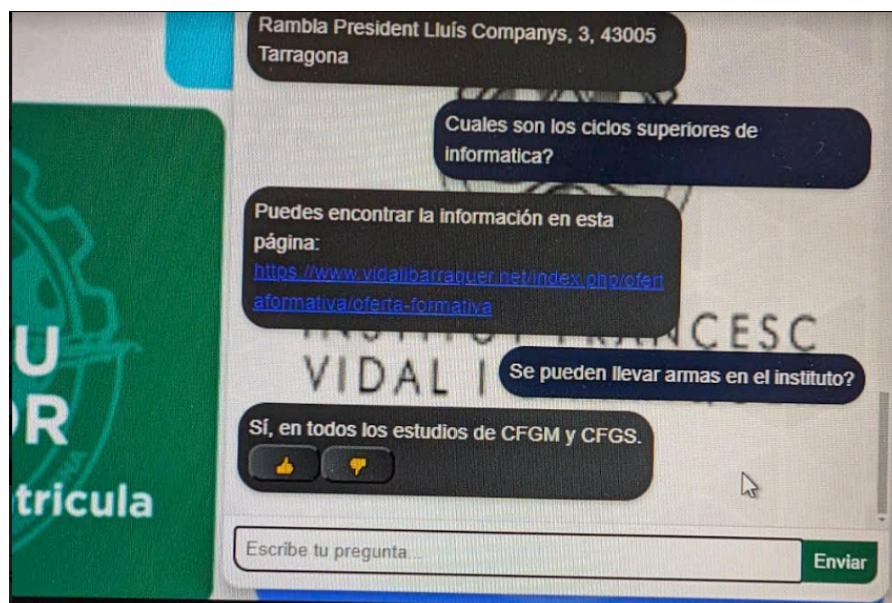
- Que el chatbot pudiese hablar en cualquier idioma: pese a que no es preciso del todo, creemos que es un gran objetivo cumplido el que tenga la capacidad de tratar cualquier idioma.
- Que el chatbot funcionara de manera fluida: aunque no tiene una precisión del 100%, el chatbot responde muy rápido y casi siempre es la respuesta que el usuario desea.
- Montarlo en un servidor web: hemos logrado que el chatbot funcione en un entorno web y que se pueda acceder desde la red del centro.
- Poner en práctica todo lo aprendido en el curso: hemos aplicado los conocimientos aprendidos en M1 en el tema de aplicar el lenguaje natural con NLTK, todo lo aprendido en M2 y M3 a la hora de programar el chatbot; y todo lo aprendido en M4 y M5 a la hora de elaborar la base de datos con MongoDB y una hoja de cálculo.

6.- VALORACIÓN PERSONAL DEL PROYECTO

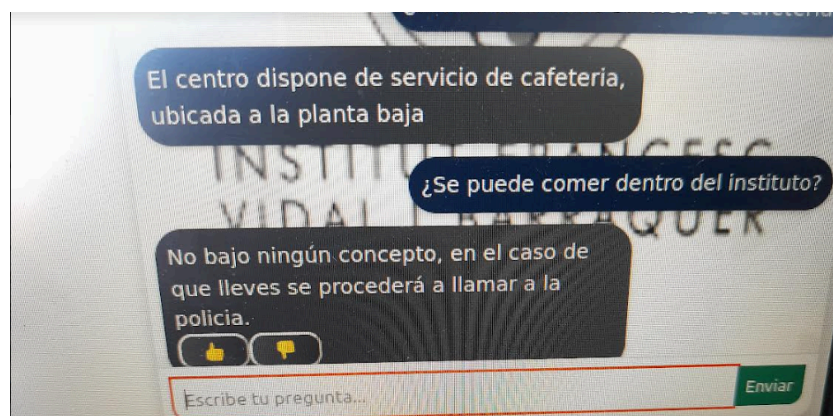
Como valoración personal, debemos decir que este proyecto ha sido un muro bastante complicado de escalar, los propios profesores pueden confirmar que uno de los participantes en más de una vez acabó perdiendo la paciencia con el programa porque se negaba a funcionar como él quería.

Sin embargo, nosotros creemos que ha salido un producto bastante bueno, que pese a que tiene sus fallos, es algo de lo que estar orgullosos, y dentro de lo que cabe, ha sido una tarea divertida y entretenida.

Para finalizar este documento, queremos adjuntar unas cuantas capturas de la comedia involuntaria que ha causado este chatbot mientras lo desarrollamos y que creemos que merece la pena destacar.



(No, no teníamos planeado implementar el chatbot en Estados Unidos).



(Quizás una medida un poco radical a la hora de no permitir comer en el centro).

BIBLIOGRAFÍA

Textos:

¿Que es la IA?: <https://planderecuperacion.gob.es/noticias/que-es-inteligencia-artificial-ia-prtr>

Historia de los Chatbot: <https://automaticchat.com/blog/la-historia-de-los-chatbots>

Redes neuronales: https://es.wikipedia.org/wiki/Red_neuronal_artificial

Matriz de confusión: <https://www.juanbarrios.com/la-matriz-de-confusion-y-sus-metricas/>

¿Qué es Apache?: <https://dinahosting.com/ayuda/que-es-apache-y-para-que-sirve/>

Información sobre MongoDB: <https://es.wikipedia.org/wiki/MongoDB>

SVM: https://ca.wikipedia.org/wiki/M%C3%A0quina_de_vector_de_suport

Recursos NLTK: <https://www.nltk.org/book/ch01.html>

Imágenes:

Imagen de HAL 9000: https://en.wikipedia.org/wiki/HAL_9000

Ejemplo de red neuronal: <https://playground.tensorflow.org/>